

Week 1 Submission

End-to-End Setup & Core CRUD Integration — Inventory Management Module

Track	Duration	Format	Stack
Full Stack	Week 1 of 4	Practical Build	React + Express + MongoDB

Task Summary

This submission implements the Week 1 evaluation task: building a functional full-stack inventory management module from scratch. A user can submit a form on the frontend to create a new inventory item; the record is persisted to a MongoDB database via a REST API, and the updated list is fetched and displayed immediately — surviving a full page refresh.

Repository

github.com/Rahmatsoga/week1-task → Week1_Task/

1. What Was Implemented

A two-tier full-stack application was built from scratch: an Express/MongoDB REST API (**server/**) and a React (Vite) client (**client/**), communicating over HTTP. The following requirements from the task brief were completed:

- Backend server with a MongoDB database connection (Mongoose) and a frontend client, organized in a clear repository structure.
- A unified feature where a user submits a frontend form to create a new inventory item.
- The record is stored in MongoDB and immediately fetched and displayed on the client — no manual refresh required.
- Full CRUD support at the API layer (Create, Read, Update, Delete), even though only Create/Read/Delete are wired into this week's UI, per the task scope.
- Schema-level validation (required fields, minimum lengths, non-negative numbers, unique SKU).
- Centralized error handling returning consistent JSON shapes and correct HTTP status codes (200/201/400/404/409/500).
- CORS correctly configured so the client (port 5173) can call the API (port 5000).
- Client-side form validation, loading states, and error banners.
- A seed script to populate sample data for immediate testing.

Evaluation Criteria Coverage

Criterion	Marks	Status
CRUD Frontend-Backend Integration	7	Complete
Database Operations	5	Complete
API Communication	5	Complete
State Management	4	Complete
Project Structure	4	Complete

2. How It Was Implemented

2.1 Project Structure

The repository separates concerns cleanly between backend and frontend, and within each, by responsibility:

```
server/  
  config/db.js – MongoDB connection logic  
  models/Item.js – Mongoose schema and validation rules  
  controllers/itemController.js – CRUD business logic  
  routes/itemRoutes.js – URL-to-controller mapping
```

```
middleware/ – centralized error handling, async wrapper
seed/seed.js – sample data generator
server.js – application entry point

client/src/
  services/itemService.js – Axios API layer
  hooks/useItems.js – state management (data, loading, error)
  components/ – ItemForm, ItemTable, Spinner
  pages/InventoryPage.jsx – assembles the full screen
```

2.2 Database Layer

A Mongoose schema (**models/Item.js**) defines the shape of an inventory item: name, sku, category, quantity, price, plus automatic createdAt/updatedAt timestamps. Validation is enforced at the schema level — required fields, minimum string lengths, non-negative numeric values, and a unique constraint on SKU — so invalid data is rejected before it ever reaches the database.

2.3 API Layer

Five REST endpoints were implemented under `/api/items`: `GET /`, `GET /:id`, `POST /`, `PUT /:id`, and `DELETE /:id`. Every response follows a consistent shape — `{ success, data, error }` — and a centralized error-handling middleware normalizes Mongoose validation errors, bad ObjectIds, and duplicate-key errors into the correct HTTP status codes (400 for validation, 404 for not-found, 409 for duplicate SKU, 500 for unexpected failures).

2.4 Frontend Integration & State Management

The `useItems` custom hook centralizes all data-fetching and mutation logic, exposing clean state (`items`, `isLoading`, `error`, `isSubmitting`) to the UI. After every create or delete action, the hook re-fetches the list from the server, guaranteeing the UI always reflects the true database state rather than relying on assumptions about what changed. The form performs client-side validation before any network call is made, and the table/spinner/error banner respond to loading and error state automatically.

2.5 Environment & CORS Configuration

Both the client and server use `.env` files (excluded from version control via `.gitignore`) for environment-specific configuration — the MongoDB connection string, allowed CORS origin, API port, and the client's API base URL. The Express `cors` middleware is configured to only accept requests from the client's origin.

3. Demo Walkthrough & Evidence

The steps below were followed to verify the reviewer checkpoint for Week 1: *"Reviewer can create a record from the UI and see it persist after refresh."*

3.1 Backend running and connected to MongoDB

The Express server starts cleanly, connects to a local MongoDB instance, and confirms the connection in the console:

```
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on http://localhost:5000
MongoDB connected: 127.0.0.1/inventory_db
```

Server running on port 5000 with MongoDB connection confirmed.

3.2 Sample data seeded

The seed script (`npm run seed`) populates 5 sample inventory items so the app can be tested immediately without manual data entry:

```
PS C:\Users\Rahmatullah\Downloads\week1 task> cd server
PS C:\Users\Rahmatullah\Downloads\week1 task\server> npm run seed

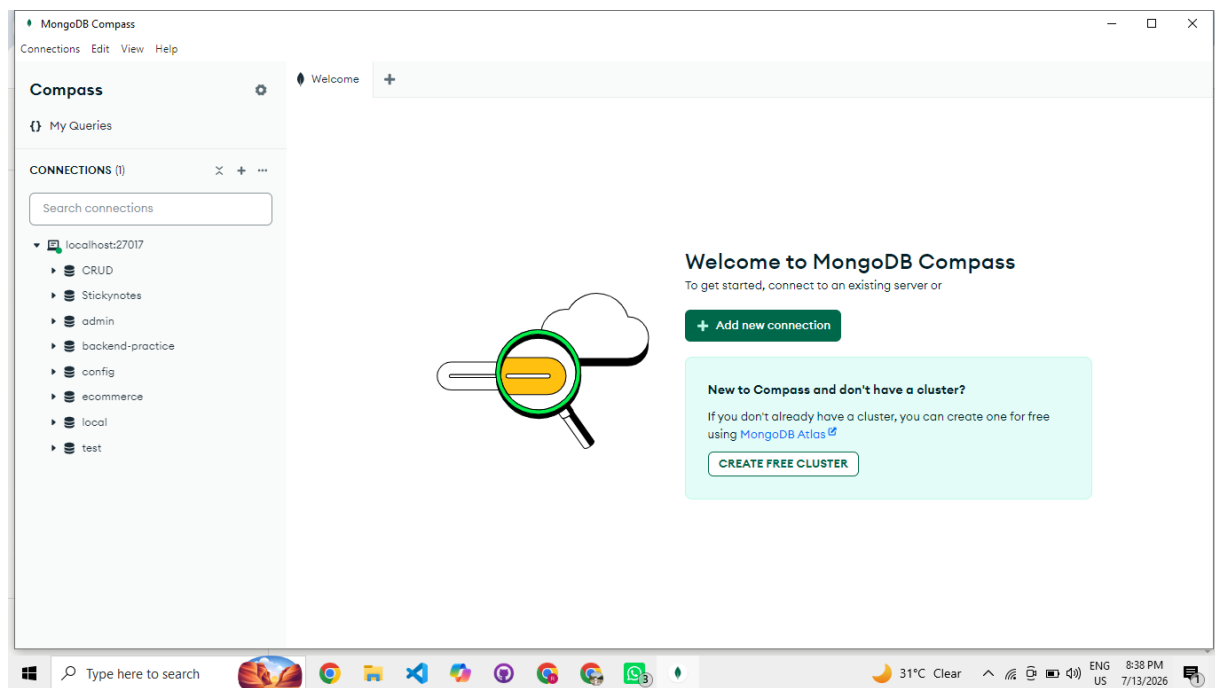
> inventory-server@1.0.0 seed
> node seed/seed.js

MongoDB connected: 127.0.0.1/inventory_db
Seeded 5 inventory items.
PS C:\Users\Rahmatullah\Downloads\week1 task\server>
```

Seed script output confirming 5 sample inventory items inserted.

3.3 Data persisted in MongoDB

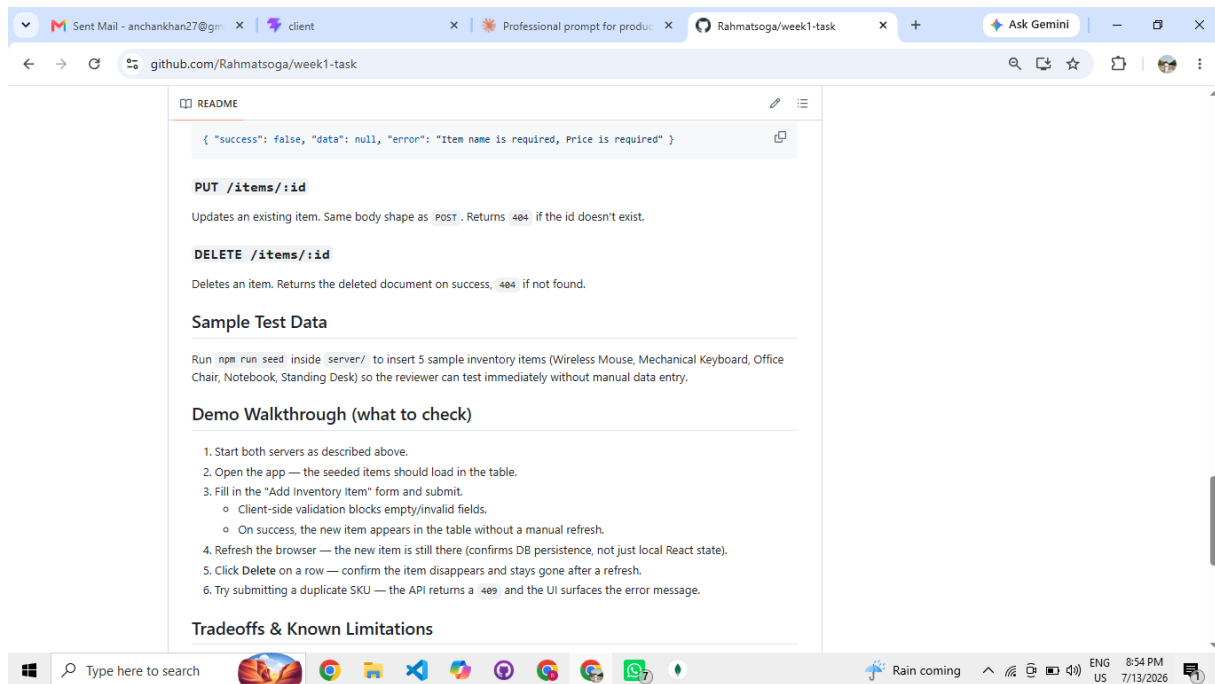
MongoDB Compass confirms the data is genuinely persisted in the `inventory_db` database, not just held in frontend memory:



MongoDB Compass showing the connected local database instance.

3.4 Working frontend — Inventory Management UI

The React client renders the "Add Inventory Item" form with client-side validation, and the "Current Inventory" table listing items fetched from the API:



Inventory Management UI: create form and live inventory table.

3.5 Verified behavior

- Submitting the form with valid data creates a new item and it appears in the table immediately, without a manual page refresh.
- Refreshing the browser (F5) confirms the new item is still present, proving persistence in MongoDB rather than local React state.
- Deleting an item removes it from the table and it remains gone after a refresh.
- Submitting a duplicate SKU correctly returns a 409 response, surfaced to the user as an error message.

4. Repository & Version Control Notes

The project is version-controlled with Git and hosted on GitHub. During setup, `.gitignore` was found to be missing from an early commit, which had caused `node_modules/` and `.env` files to be committed by mistake. This was corrected by adding proper `.gitignore` files to both `client/` and `server/`, and removing those files from Git tracking (`git rm --cached`) while keeping them intact on disk locally. Environment secrets are never committed; only `.env.example` templates are tracked.

README Documentation

The repository includes a comprehensive README covering project structure, setup instructions, environment variables, full API documentation with example requests/responses, sample test data instructions, a demo walkthrough checklist, and a tradeoffs/limitations section:

```
PS C:\Users\Rahmatullah\Downloads\week1 task> git ls-files | Select-String ".env"

client/.env
client/node_modules/@babel/core/lib/config/helpers/environment.js
client/node_modules/@babel/core/lib/config/helpers/environment.js.map
client/node_modules/axios/lib/env/README.md
client/node_modules/axios/lib/env/classes/FormData.js
client/node_modules/axios/lib/env/data.js
client/node_modules/caniuse-lite/data/features/css-env-function.js
client/node_modules/node-releases/data/processed/envs.json
client/node_modules/proxy-from-env/LICENSE
client/node_modules/proxy-from-env/README.md
client/node_modules/proxy-from-env/index.cjs
client/node_modules/proxy-from-env/index.js
client/node_modules/proxy-from-env/package.json
client/node_modules/vite/dist/client/env.mjs
server/node_modules/dotenv/CHANGELOG.md
server/node_modules/dotenv/LICENSE
server/node_modules/dotenv/README-es.md
server/node_modules/dotenv/README.md
server/node_modules/dotenv/SECURITY.md
server/node_modules/dotenv/config.d.ts
server/node_modules/dotenv/config.js
server/node_modules/dotenv/lib/cli-options.js
server/node_modules/dotenv/lib/env-options.js
server/node_modules/dotenv/lib/main.d.ts
server/node_modules/dotenv/lib/main.js
server/node_modules/dotenv/package.json
server/node_modules/mquery/lib/env.js
```

Excerpt of the project README on GitHub.

5. Tradeoffs & Known Limitations

- No authentication yet — this is the Week 2 deliverable. All endpoints are currently public.
- PUT (update) is implemented on the backend and in the API service layer, but there is no edit UI yet this week — only create/read/delete are wired into the interface, per the Week 1 task scope.
- Pagination and search (Week 3 scope) are not implemented yet; GET /items returns the full collection, which is fine for the current dataset size.
- Using `window.confirm` for delete confirmation is functional but minimal; a proper modal would be a nicer UX in a production build.

6. Improvements With More Time

- Add optimistic UI updates on create/delete instead of a full re-fetch.

- Add unit tests (Jest + React Testing Library on the client, Supertest on the server).
- Add request-body validation middleware (e.g. express-validator) as a defense-in-depth layer in front of Mongoose schema validation.
- Dockerize both services with a docker-compose.yml including MongoDB.